

Prolog Debugging Tutorial

1. Prolog Debugging in Trace Mode

To start the debugger in the trace mode, you can use the following commands:

- **trace.** - This command will start the debugger in trace mode. In trace mode, the debugger will stop at every predicate call, allowing you to step through the program flow.
- **debug.** - This command enables debugging mode without trace mode, but it enables spy point mode instead.
- **nodebug.** - This command turns off both debugging in trace mode and spy point mode. In this case one will run Prolog programs without debugging facility.

2. Understanding Debugging with the Box Model

Please refer to Chapter 8 pages 194 – 200 of “Programming in Prolog, 5th Edition” textbook.

When you run the debugger, you will encounter different event which represents different stages of the query evaluation:

- **Call:** The goal is about to be solved.
- **Exit:** The goal has been successfully solved.
- **Redo:** The goal is being re-evaluated (backtracking).
- **Fail:** The goal has failed.

Example:

```
child(abraham,ishmael).  
child(abraham,isaac).  
child(isaac,esau).
```

```
descendant(X,Y) :- child(X,Y).  
descendant(X,Z) :- child(X,Y), descendant(Y,Z).
```

3. Stepping Through Code

While in the debugger, you can use the following commands to control execution:

- **c (creep):** Step into evaluating the goal.
- **s (skip):** Skip over the current goal.
- **r (retry):** Retry the current goal (backtrack).
- **a (abort):** Abort evaluation at that point and return to the top level.
- **f (fail):** Force the current goal to fail.

Write the codes in the Prolog editor:

```

descendant.pl [modified]
File Edit Browse Compile Prolog Pce Help
descendant.pl [modified]
child(abraham,ishmael).
child(abraham,isaac).
child(isaac,esau).

descendant(X,Y) :- child(X,Y).
descendant(X,Z) :- child(X,Y),descendant(Y,Z).

```

Colourising buffer ... done, 0.00 seconds, 20 fragments Line: 8

Run codes without debugger.

```

?- child(X,Y).
X = abraham,
Y = ishmael ;
X = abraham,
Y = isaac ;
X = isaac,
Y = esau.

?- descendant(X,Y).
X = abraham,
Y = ishmael ;
X = abraham,
Y = isaac ;
X = isaac,
Y = esau ;
X = abraham,
Y = esau ;
false.

```

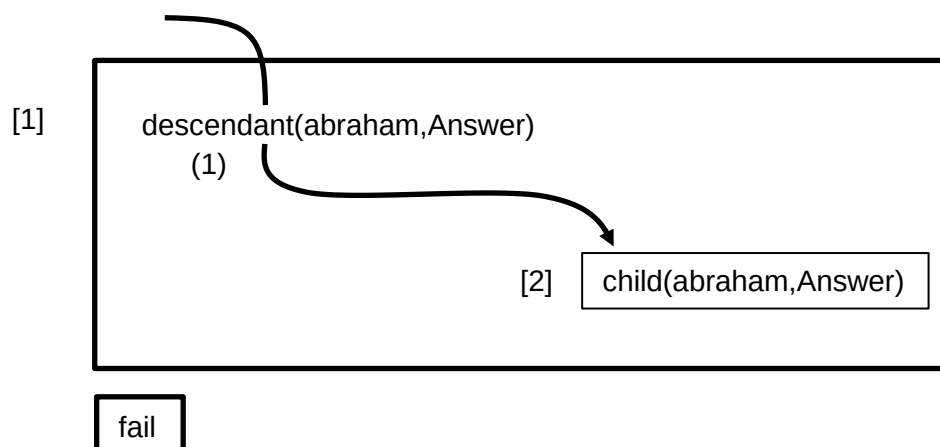
Turn on the trace mode:

```
?- trace.
```

Then run this query.

```
? - descendant(abraham,Answer),fail.
```

Here we use fail to force Prolog to backtrack all possible answers, this is essentially equivalent to pressing space or type ;



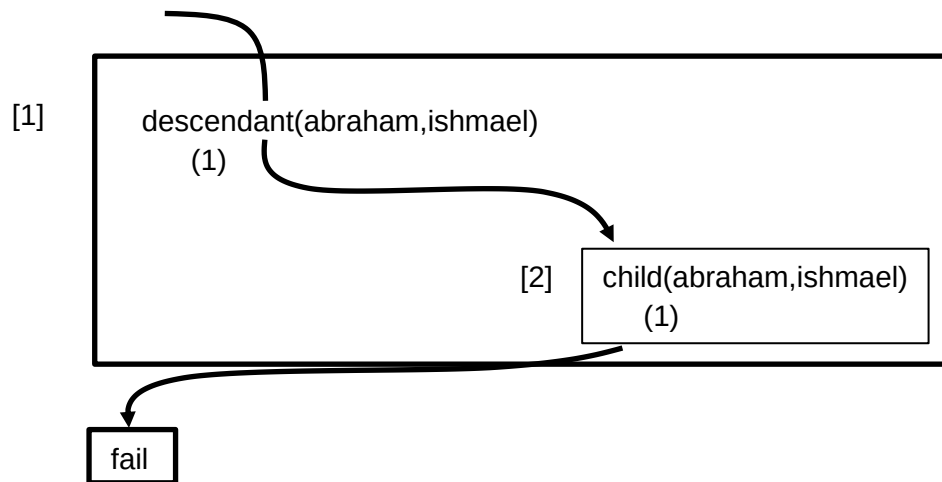
```

[trace] ?- descendant(abraham, Answer), fail.
Call: (13) descendant(abraham, _30178) ? creep
Call: (14) child(abraham, _30178) ? creep
Exit: (14) child(abraham, ishmael) ? creep
Exit: (13) descendant(abraham, ishmael) ? creep

```

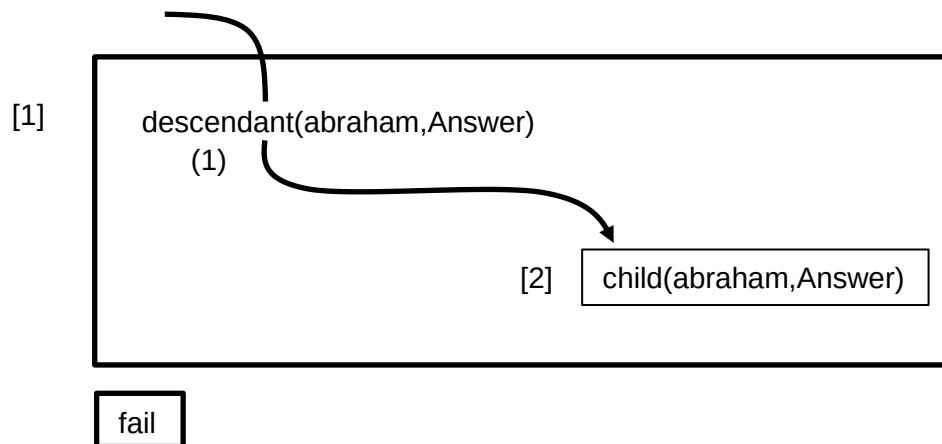
Answer is ishmael.

Press space or ; to creep



We encountered fail, so Prolog will redo

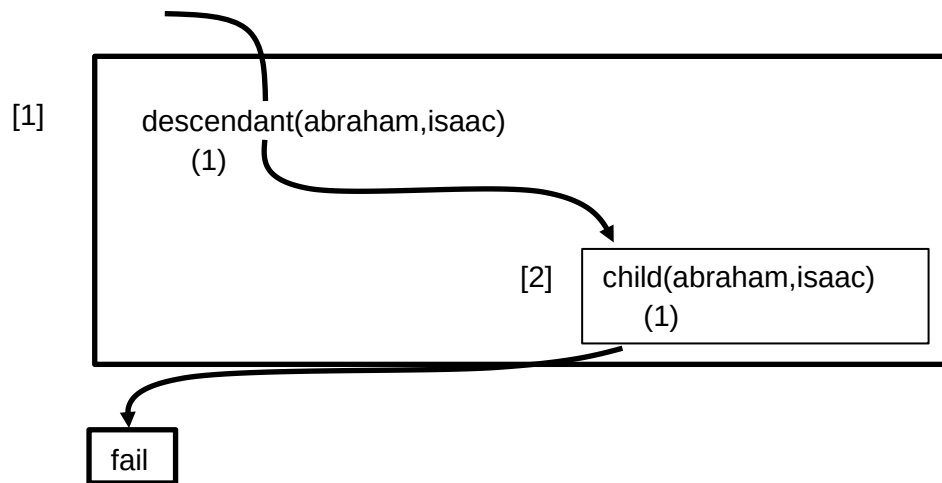
```
Fail: (13) fail ? creep
Redo: (14) child(abraham, _30178) ? creep
```



So now the arrow retreated back to the child predicate.

```
Exit: (14) child(abraham, isaac) ? creep
Exit: (13) descendant(abraham, isaac) ? creep
```

And now we find another answer (which is isaac)



Then Fail forced Prolog to redo the goal

```

Call: (13) fail ? creep
Fail: (13) fail ? creep
  
```

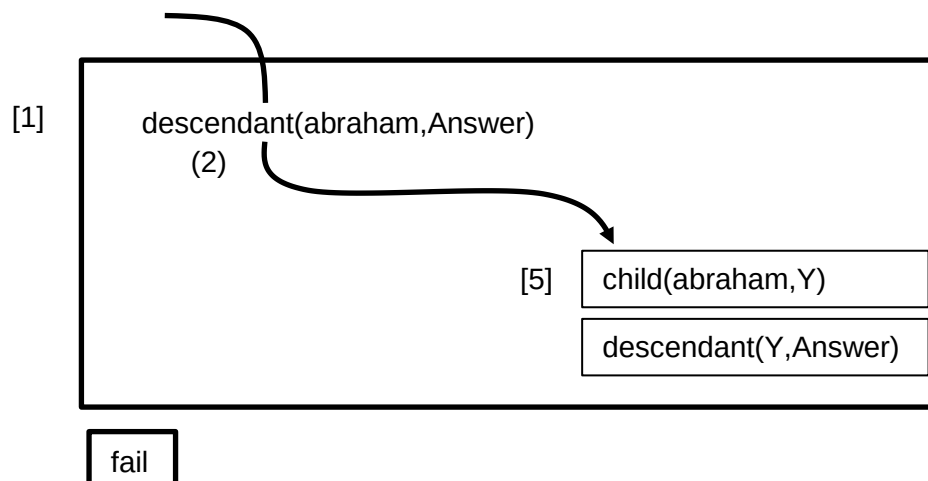
Now, since using only `child` facts cannot offer us any more solution, Prolog chooses another rule:

```

descendant(X,Z) :- child(X,Y), descendant(Y,Z).
  
```

```

Redo: (13) descendant(abraham, _30178) ? creep
Call: (14) child(abraham, _40460) ? creep
Exit: (14) child(abraham, ishmael) ? creep
Call: (14) descendant(ishmael, _30178) ? creep
  
```



```

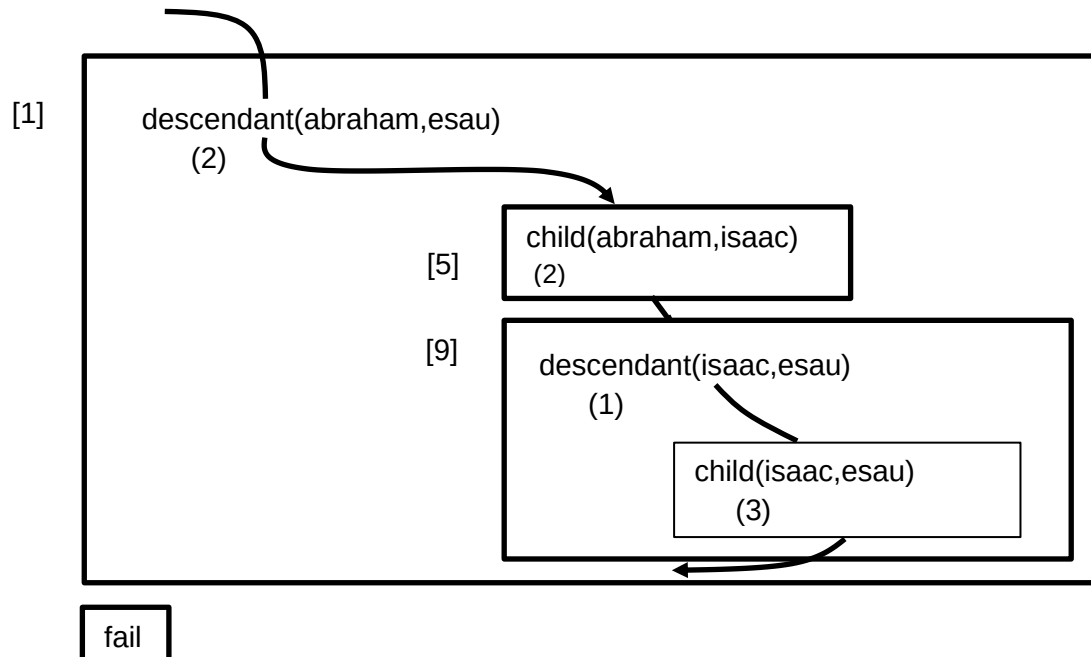
Call: (15) child(ishmael, _30178) ? creep
Fail: (15) child(ishmael, _30178) ? creep
Redo: (14) descendant(ishmael, _30178) ? 
  
```

Since ishmael has no child, the goal fails and Prolog goes back to find alternatives

```

Fail: (15) child(ishmael, _45322) ? creep
Fail: (14) descendant(ishmael, _30178) ? creep
Redo: (14) child(abraham, _40460) ? creep
Exit: (14) child(abraham, isaac) ? creep
Call: (14) descendant(isaac, _30178) ? creep
Call: (15) child(isaac, _30178) ? creep
Exit: (15) child(isaac, esau) ? creep
Exit: (14) descendant(isaac, esau) ? creep
Exit: (13) descendant(abraham, esau) ? creep
Call: (13) fail ? creep
Fail: (13) fail ? creep

```



This is the full trace.

```

[trace] ?- descendant(abraham, Answer), fail.
Call: (13) descendant(abraham, _16298) ? creep
Call: (14) child(abraham, _16298) ? creep
Exit: (14) child(abraham, ishmael) ? creep
Exit: (13) descendant(abraham, ishmael) ? creep
Call: (13) fail ? creep
Fail: (13) fail ? creep
Redo: (14) child(abraham, _16298) ? creep
Exit: (14) child(abraham, isaac) ? creep
Exit: (13) descendant(abraham, isaac) ? creep
Call: (13) fail ? creep
Fail: (13) fail ? creep
Redo: (13) descendant(abraham, _16298) ? creep
Call: (14) child(abraham, _26580) ? creep
Exit: (14) child(abraham, ishmael) ? creep
Call: (14) descendant(ishmael, _16298) ? creep
Call: (15) child(ishmael, _16298) ? creep
Fail: (15) child(ishmael, _16298) ? creep
Redo: (14) descendant(ishmael, _16298) ? creep
Call: (15) child(ishmael, _31442) ? creep
Fail: (15) child(ishmael, _31442) ? creep
Fail: (14) descendant(ishmael, _16298) ? creep
Redo: (14) child(abraham, _26580) ? creep
Exit: (14) child(abraham, isaac) ? creep
Call: (14) descendant(isaac, _16298) ? creep
Call: (15) child(isaac, _16298) ? creep
Exit: (15) child(isaac, esau) ? creep
Exit: (14) descendant(isaac, esau) ? creep
Exit: (13) descendant(abraham, esau) ? creep
Call: (13) fail ? creep
Fail: (13) fail ? creep
Redo: (14) descendant(isaac, _16298) ? creep
Call: (15) child(isaac, _41950) ? creep
Exit: (15) child(isaac, esau) ? creep
Call: (15) descendant(esau, _16298) ? creep
Call: (16) child(esau, _16298) ? creep
Fail: (16) child(esau, _16298) ? creep
Redo: (15) descendant(esau, _16298) ? creep
Call: (16) child(esau, _46812) ? creep
Fail: (16) child(esau, _46812) ? creep
Fail: (15) descendant(esau, _16298) ? creep
Fail: (14) descendant(isaac, _16298) ? creep
Fail: (13) descendant(abraham, _16298) ? creep
false.

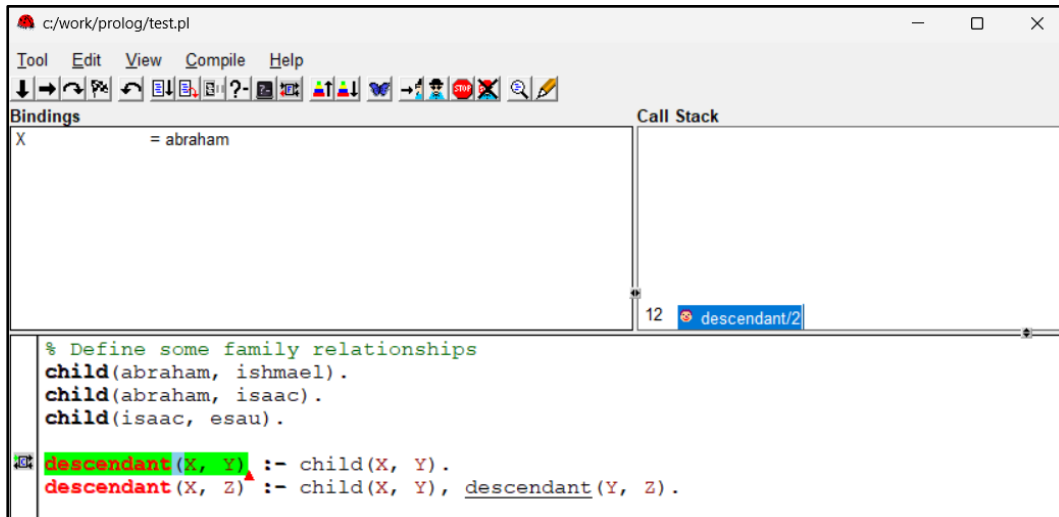
```

4. Debugging in Graphic Tracing Mode

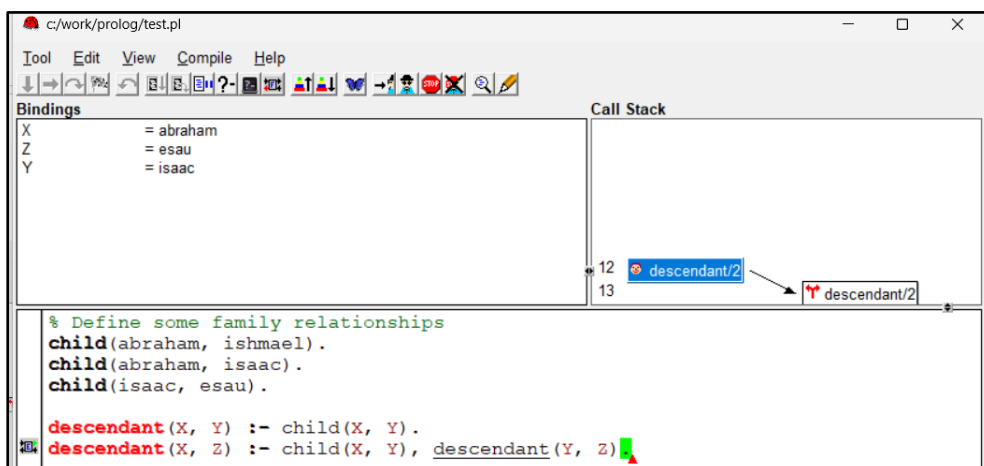
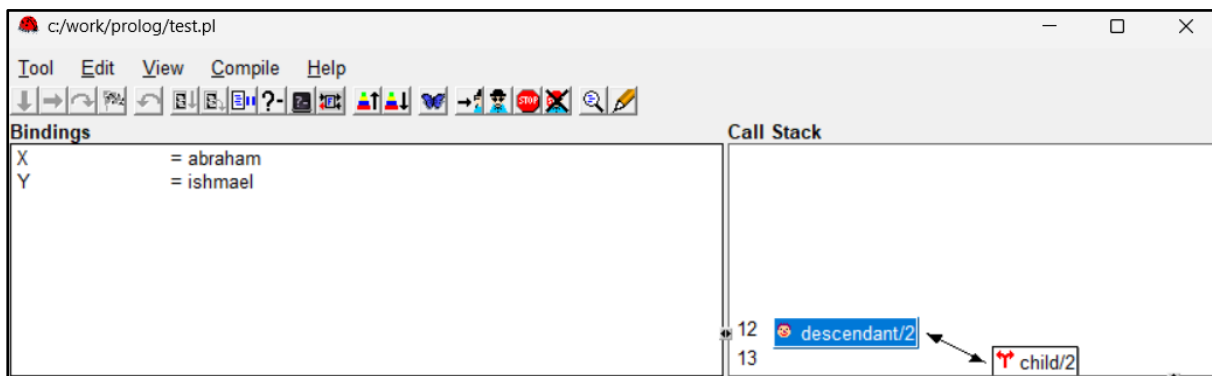
In SWI-Prolog, there's also a graphical tracer, which provides information about call stack and variables. You can enable it using **gtrace**.

```
?- gtrace.  
% The graphical front-end will be used for subsequent tracing  
true.
```

Try using **gtrace** with the previous query.



You can inspect the call stack, and variables.



**** Note that you may need to use the fail predicate, or switch back to the swi-pl window and press space in order to get more answers.**

5. Debugging Using Spy Points

Spy points allow you to specify specific predicates where you want the debugger to stop. This is useful if you want to focus on certain parts of your code.

- **spy(Predicate/Arity)** . - Set a spy point on a specific predicate.
- **nospy(Predicate/Arity)** . - Remove the spy point.

Copy this code:

```
my_member(X, [X|_]) .  
my_member(X, [_|Xs]) :- my_member(X, Xs) .  
  
?- spy(my_member/2) .  
% Spy point on my_member/2 established.
```

When you run your code, the debugger will stop whenever the `member/2` predicate is called.

5. Removing Spy Points

You can remove a spy point using the `nospy` command:

```
?- nospy(my_member/2) .  
% Spy point on my_member/2 removed.
```

You can also remove all spy points at once:

```
?- nospyall .  
% All spy points removed.
```

6. Interactive Debugging Example

Here's a simple example using the `member/2` predicate:

```
?- spy(my_member/2) .  
?- my_member(X, [1,2,3,4,5]) .
```

Output:

```

?- debug
|
true.

[debug] ?- spy(my_member).
% Spy point on my_member/2
true.

[debug] ?- my_member(X, [1,2,3,4,5]).
Call: (12) my_member(_18342, [1, 2, 3, 4, 5]) ? creep
Exit: (12) my_member(1, [1, 2, 3, 4, 5]) ? creep
X = 1 ;
Redo: (12) my_member(_18342, [1, 2, 3, 4, 5]) ? creep
Call: (13) my_member(_18342, [2, 3, 4, 5]) ? creep
Exit: (13) my_member(2, [2, 3, 4, 5]) ? creep
Exit: (12) my_member(2, [1, 2, 3, 4, 5]) ? creep
X = 2 ;
Redo: (13) my_member(_18342, [2, 3, 4, 5]) ? creep
Call: (14) my_member(_18342, [3, 4, 5]) ? creep
Exit: (14) my_member(3, [3, 4, 5]) ? creep
Exit: (13) my_member(3, [2, 3, 4, 5]) ? creep
Exit: (12) my_member(3, [1, 2, 3, 4, 5]) ? creep
X = 3 ;
Redo: (14) my_member(_18342, [3, 4, 5]) ? creep
Call: (15) my_member(_18342, [4, 5]) ? creep
Exit: (15) my_member(4, [4, 5]) ? creep
Exit: (14) my_member(4, [3, 4, 5]) ? creep
Exit: (13) my_member(4, [2, 3, 4, 5]) ? creep
Exit: (12) my_member(4, [1, 2, 3, 4, 5]) ? creep
X = 4 ;
Redo: (15) my_member(_18342, [4, 5]) ? creep
Call: (16) my_member(_18342, [5]) ? creep
Exit: (16) my_member(5, [5]) ? creep
Exit: (15) my_member(5, [4, 5]) ? creep
Exit: (14) my_member(5, [3, 4, 5]) ? creep
Exit: (13) my_member(5, [2, 3, 4, 5]) ? creep
Exit: (12) my_member(5, [1, 2, 3, 4, 5]) ? creep
X = 5 ;
Redo: (16) my_member(_18342, [5]) ? creep
Call: (17) my_member(_18342, []) ? creep
Fail: (17) my_member(_18342, []) ? creep
Fail: (16) my_member(_18342, [5]) ? creep
Fail: (15) my_member(_18342, [4, 5]) ? creep
Fail: (14) my_member(_18342, [3, 4, 5]) ? creep
Fail: (13) my_member(_18342, [2, 3, 4, 5]) ? creep
Fail: (12) my_member(_18342, [1, 2, 3, 4, 5]) ? creep
false

```

The debugger stops at each stage of the `my_member/2` predicate execution, allowing you to examine the flow of query evaluation on this predicate.